Выбор метрики

Метрика является функцией, задающей расстояние в метрическом пространстве, и должна удовлетворять следующим аксиомам:

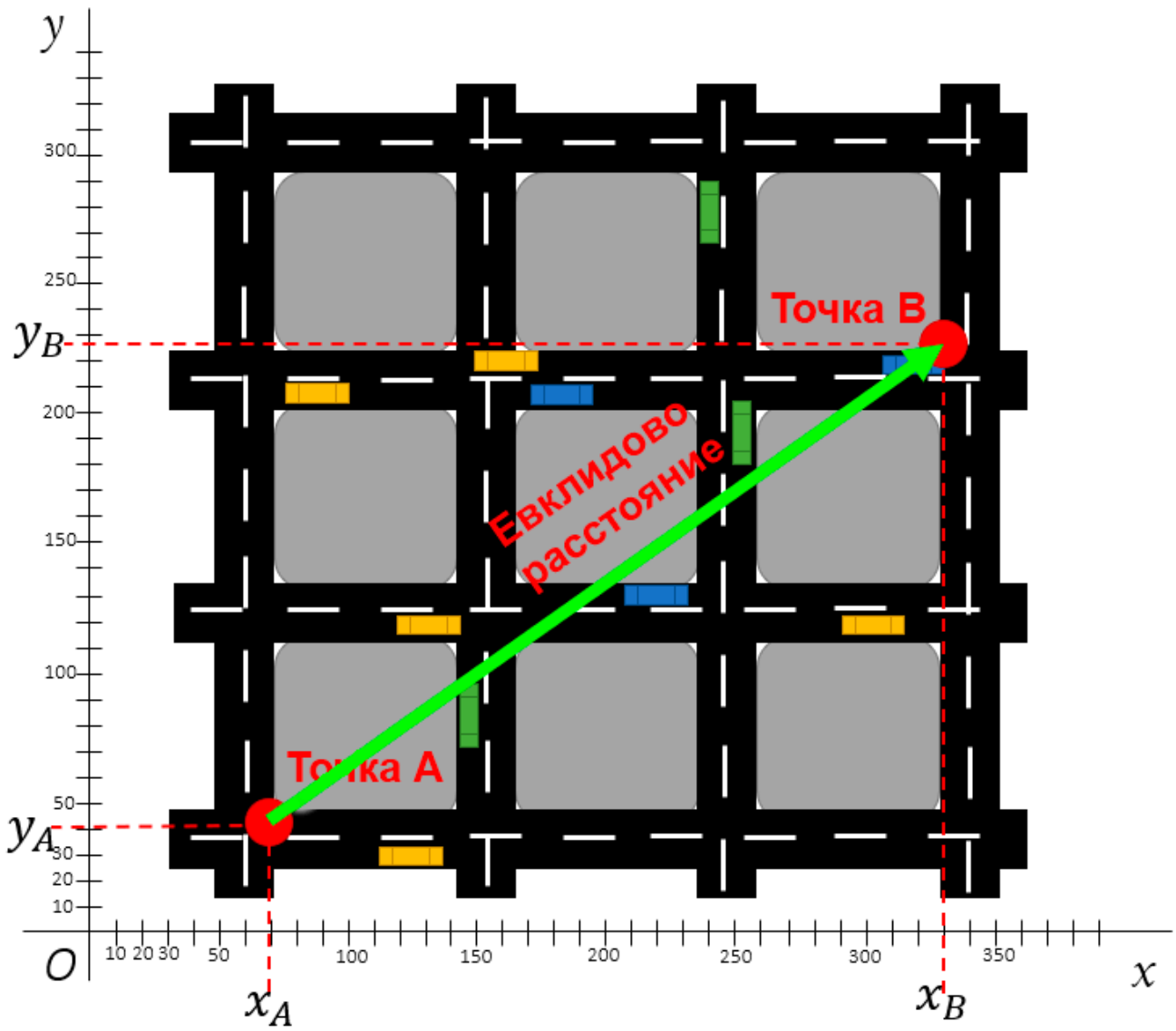
1. $\rho(x, y) >= 0$, причем $\rho(x, y) = 0 \equiv x = y$
2. $\rho(x, y) = \rho(y, x)$,
3. $\rho(x, y) <= \rho(x, z) + \rho(z, y)$.

Метрики

Евклидово расстояние

Это, по-видимому, наиболее общий тип расстояния. Оно попросту является геометрическим расстоянием в многомерном пространстве и вычисляется следующим образом.

$$r(x, y) = \sqrt{(\sum_i (x_i - y_i)^2)}$$



Квадрат евклидова расстояния

Иногда может возникнуть желание возвести в квадрат стандартное евклидово расстояние, чтобы придать большие веса более отдаленным друг от друга объектам.

$$r(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$$

Взвешенное евклидово расстояние

Применяется в тех случаях, когда каждому i-свойству удается приписать некоторый «вес», пропорционально степени важности признака в задаче классификации.

$$r(x, y) = \sqrt{\sum_i w_i (x_i - y_i)^2}$$

Расстояние городских кварталов (манхэттенское расстояние).

Это расстояние является просто средним разностей по координатам. В большинстве случаев эта мера расстояния приводит к таким же результатам, как и расстояние Евклида. (L1, а обобщение – метрика Минковского Lk)

$$r(x, y) = \sum_i |x_i - y_i|$$

Метрика Минковского (обобщение Евклидовой и Манхэттенской метрик)

$$\rho(x, y) = (\sum_{i=1}^{n} |x_i - y_i|^q)^{\frac{1}{q}}$$

Примеры метрик

Процент несогласия

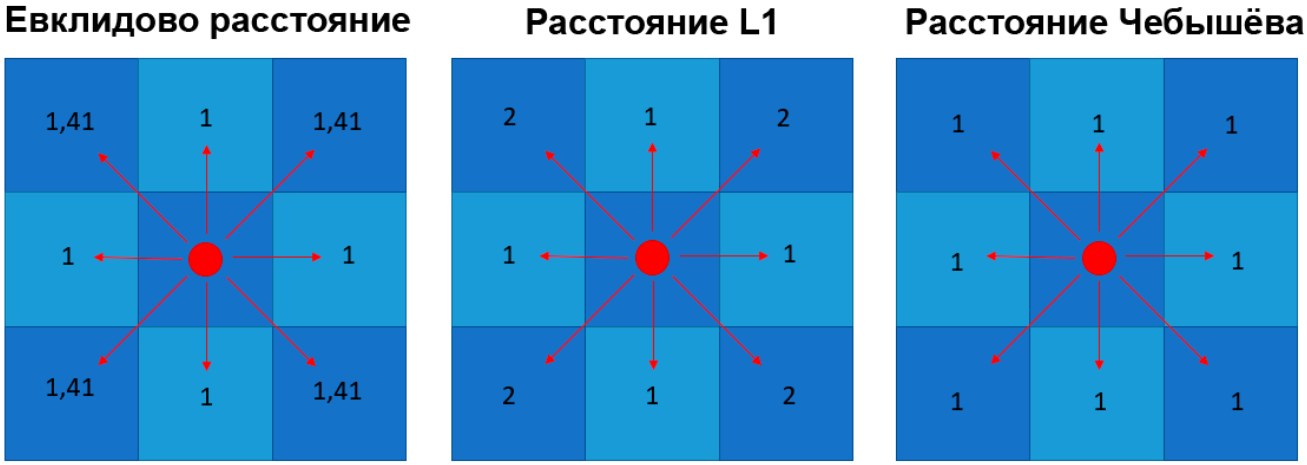
Эта мера используется в тех случаях, когда данные являются категориальными. Например, первый признак объекта – пол, второй – возраст, третий – место работы. Представим значения свойств (признаков) объекта в виде вектора значений. Первый вектор – (муж, 20 лет, учитель), второй вектор – (муж, 28 лет, менеджер). Процент несогласия равен 2/3. Эти вектора различаются на 66.6%.

$$\rho(x, y) = \frac{|\{x_i \neq y_i\}|}{n}$$

Расстояние Чебышева – метрика шахматной доски

(равно минимальному количеству ходов, которое необходимо королю, чтобы перейти из одного поля в другое).

$$\rho(x, y) = \max(x_i - y_i)$$



Другие меры близости

- Коэффициент корреляции
- Косинусная мера
- Коэффициент Жаккара
- Редакционное расстояние, расстояние Левенштейна
- Расстояния Хемминга

Нормализация признаков

Рассмотрим простой пример чувствительности метода ближайшего соседа к масштабу признаков. Допустим, решается задача определения пола человека по двум признакам: росту (в сантиметрах, принимает значения примерно от 150 до 200) и уровню экспрессии гена SRY (безразмерная величина от нуля до единицы; у мужчин ближе к единице, y женщин ближе к нулю). Обучающая выборка состоит из двух объектов: x_1 = (180, 0.2), девочка и x_2 = (173, 0.9), мальчик. Требуется классифицировать новый объект u = (178, 0.85). Воспользуемся классификатором одного ближайшего соседа. Расстояния от и до объектов обучения равны $\rho(u, x_1) = 21$ и $\rho(u, x_2) = 5$. Мы признаем новый объект девочкой, хотя это не так - высокий уровень экспрессии гена SRY позволяет с уверенностью сказать, что это мальчик. Из-за сильных различий в масштабе признаков уровень экспрессии практически не учитывается при классификации, что совершенно неправильно. Чтобы избежать подобных проблем, признаки следует нормировать.

- Нормировка на единичную дисперсию
  $x^j = \frac{x^j - x^j}{(x^j)}$
- Нормировка на отрезок $[0, 1]$:
  $x^j = \frac{x^j - min(x^j)}{max(x^j) - min(x^j))}$

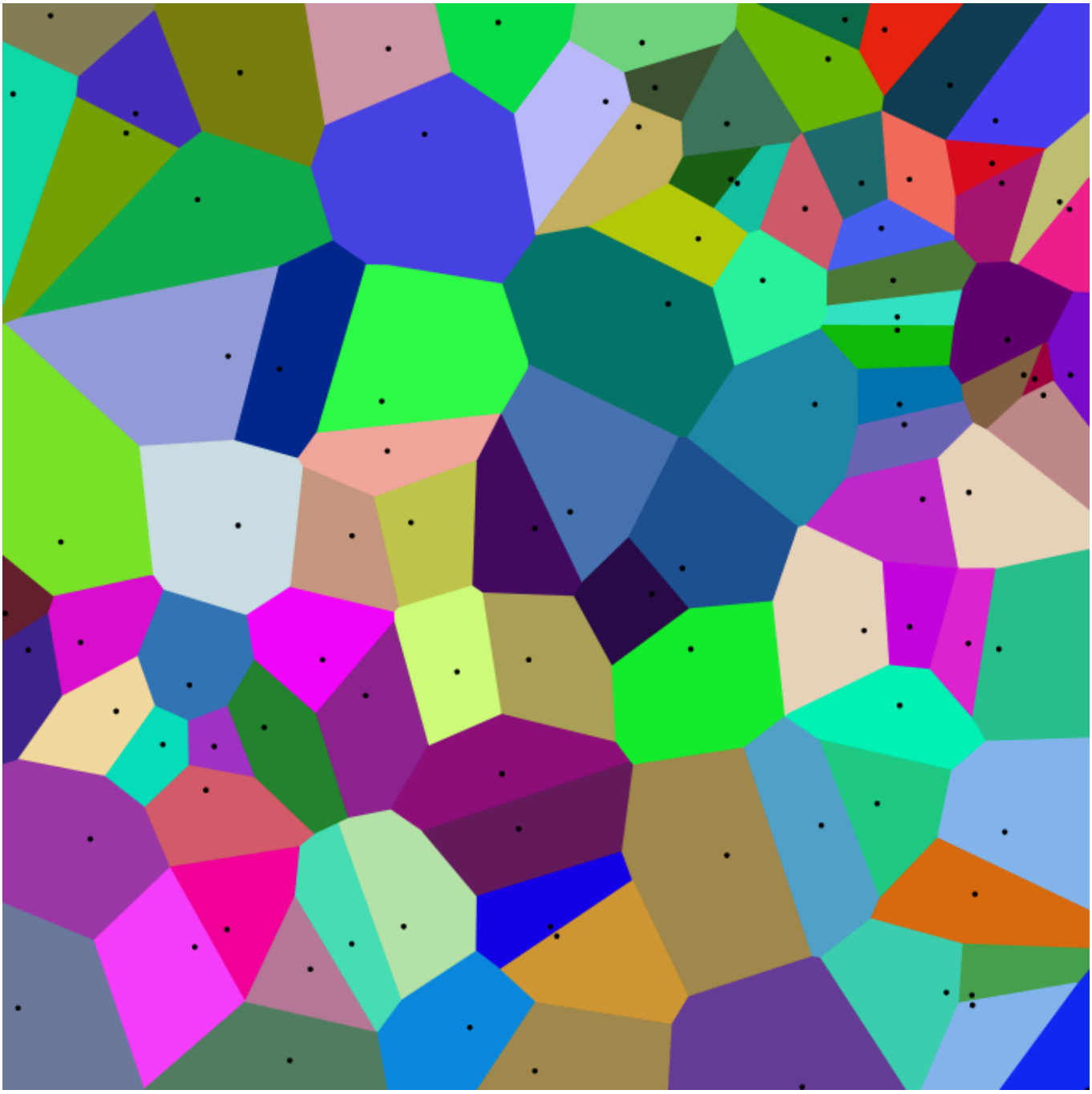
Здесь x^j — это вектор, составленный из j-х признаков всех объектов. Иными словами это j-й столбец матрицы "объекты-признаки"

Время выполнения

Серьезным ближайших соседей ограничением методов является их время выполнения. Сравнение точки q в d измерениях с п соседями выполняется в ходе n явных сравнений расстояний ценой O(nd). Из-за наличия тысяч или даже миллионов доступных точек этот поиск может внести заметную задержку в любую систему классификации. Один из подходов к ускорению поиска подразумевает использование геометрических структур данных. Наиболее популярные – это диаграммы Вороного и kd-деревья.

Диаграмма Вороного

Диаграмма Вороного визуализатор



$= \{_1, 2, \ldots, _n\}$ – множество точек на плоскости

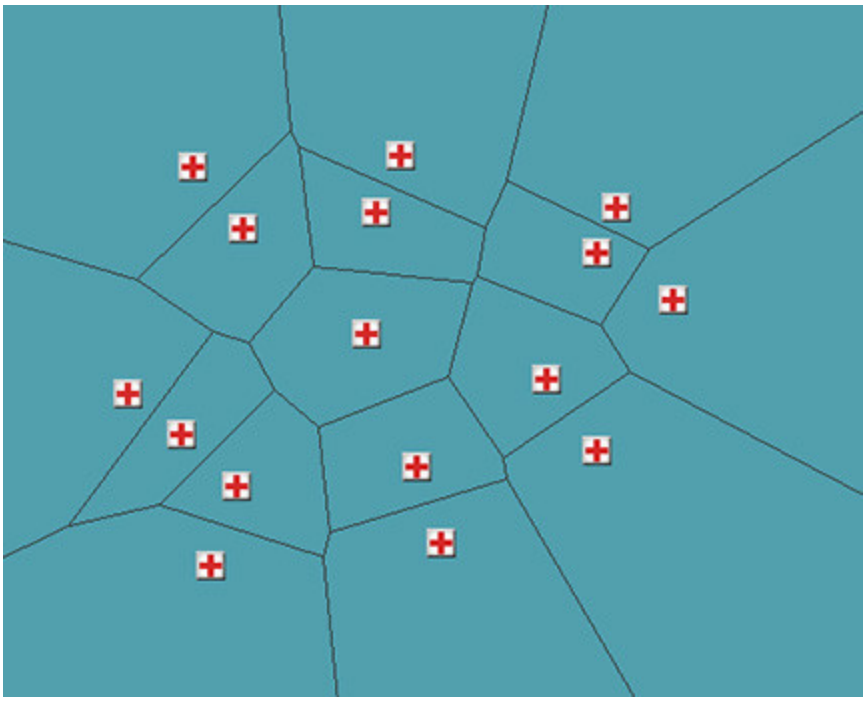
$_i \in$ называется **сайтом** (site)

Ячейка Вороного (Voronoi cell) – множество точек плоскости q таких, что для фиксированного сайта , и любых других сайтов $_j \in$ $j \neq i$ верно неравенство $\rho(q, _i) < \rho(q, _j)$.

Диаграмма Вороного (Voronoi diagram) для сайтов $= \{_1, 2, \ldots, _n\}$ на плоскости — это разбиение плоскости на ячейки Вороного для каждого сайта из . Для набора целевых точек мы бы хотели разделить пространство вокруг них на ячейки так, чтобы каждая ячейка содержала ровно одну целевую точку. Кроме того, мы хотим, чтобы целевая точка каждой ячейки была ближайшим соседом для всех местоположений в ячейке.

Пример

Пусть есть карта города, на которой точками обозначены аптеки. Если нужно срочно купить лекарство, человек пойдет в ближайшую аптеку. Ему интересно знать, какая аптека ближе, для любой точки города. Для этого он может взять карту и расчертить её на ячейки так, чтобы внутри каждой ячейки находилось только одна аптека, а для всех остальных точек ячейки именно эта аптека была ближайшей. Полученная картинка и будет диаграммой Вороного для точек-аптек.

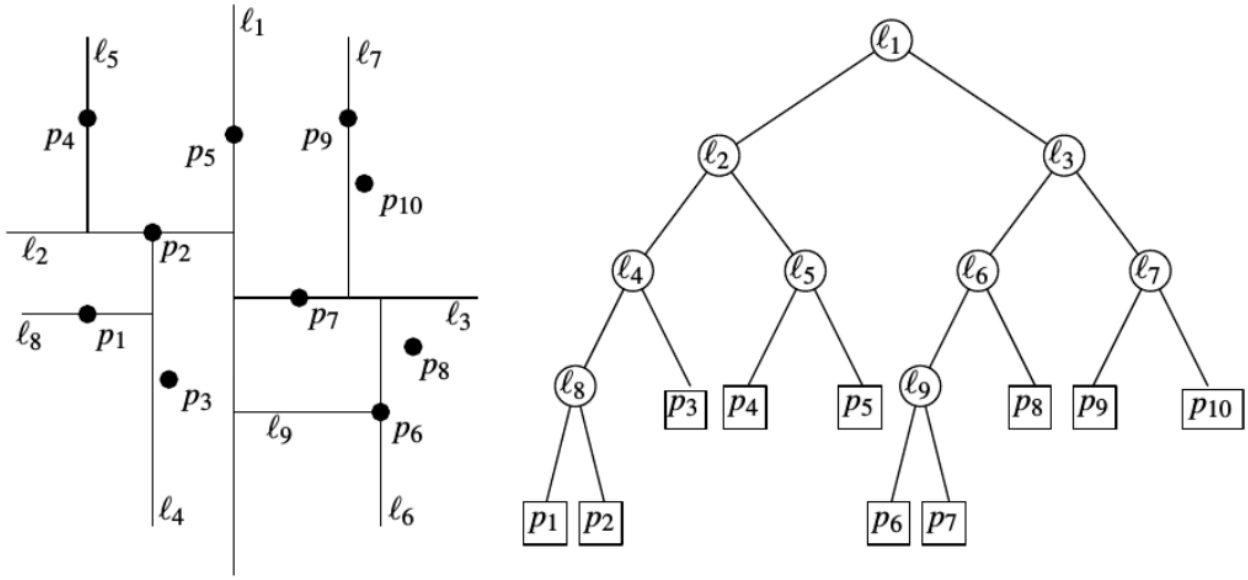


Если каждая точка обозначает аптеку, то мы довольно быстро можем определить, где находится ближайшая к нам, но сначала нужно определить область, в которой мы находимся. Диаграммы Вороного не обеспечивают эффективного способа определения области, однако для ускорения поиска мы можем хранить ссылку на каждую область в дереве квадрантов или в R-дереве. А узнав область, мы сможем узнать её соседей, и соседей её соседей.

Например, если аптечки в вашей области больше нет, то вам нужно найти путь к другой ближайшей. Зная область, мы можем узнать её ребра. А при помощи этих рёбер мы можем получить соседей. Будем брать ближайшего соседа и смотреть, есть ли в нём аптечка.

Диаграммы Вороного являются прекрасным инструментом для размышлений о данных и обладают множеством полезных свойств. Существуют эффективные алгоритмы их построения и поиска, особенно в двух измерениях. Однако эти процедуры быстро усложняются по мере увеличения размерности, что делает их, как правило, непрактичными за пределами двух или трех измерений.

Kd-деревья



Kd-дерево (short for k-dimensional tree) – статическая структура данных для хранения точек в k-мерном пространстве. Позволяет отвечать на запрос, какие точки лежат в данном прямоугольнике.

Строится это дерево следующим образом: разобьём все точки вертикальной прямой так, чтобы слева (нестрого) и справа (строго) от неё было примерно поровну точек (для этого посчитаем медиану первых координат). Получим подмножества для левого и правого ребёнка. Далее построим для этих подмножеств деревья, но разбивать будем уже не вертикальной, а горизонтальной прямой (для этого посчитаем медиану вторых координат). И так далее (будем считать, что $k = 2$ (случай больших размерностей обрабатывается аналогично), поэтому на следующем уровне вновь будем разбивать вертикальными прямыми).

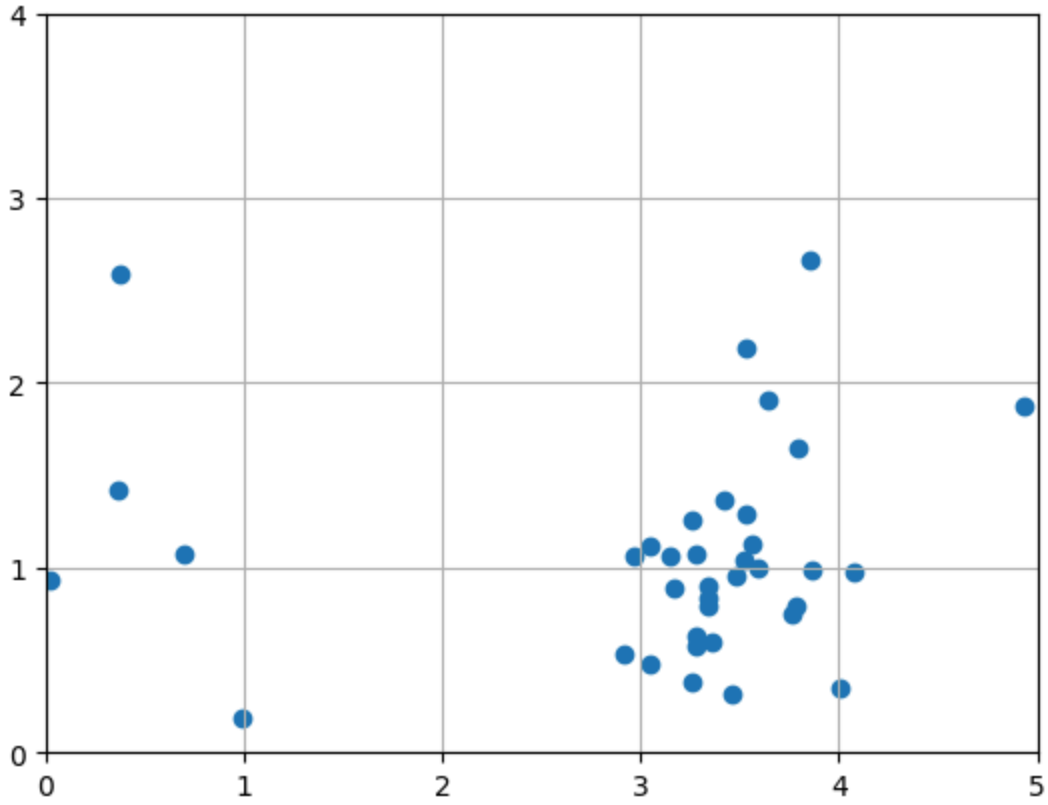
Эта конструкция иерархии идеально подходит для поддержки поиска. Начиная с корня, мы проверяем, находится ли точка запроса q слева или справа от средней линии/плоскости. Это определяет, на какой стороне лежит q, а следовательно, к какой стороне дерева следует вернуться.

Таким образом, обходя это дерево определенным образом, можно найти точку из обучения, ближайшую к заданной. Если размерность пространства небольшая (10-20), то данный подход позволяет находить ближайшего соседа за время порядка $O(\lg l)$.

Индексные сетки

Мы можем разбить пространство на d-мерные блоки, разделив: диапазон каждого измерения на г интервалов или сегментов. Рассмотрим, например, двумерное пространство, в котором каждая ось представляет собой вероятность, варьируясь, таким образом, от 0 до 1. Этот диапазон можно разделить на r равных интервалов, так что i-й интервал находится в диапазоне $[\frac{i-1}{r}, \frac{i}{r}]$. Эти интервалы определяют регулярную сетку в пространстве, поэтому мы можем связать каждую из учебных точек с ячейкой сетки, которой она принадлежит. Поиск теперь становится проблемой определения правильной ячейки сетки для точки a с помощью поиска в массиве или двоичного поиска, а затем

сравнения q со всеми точками в этой ячейке для определения ближайшего соседа. Такие индексные сетки могут быть эффективными, но есть потенциальные проблемы. В первую очередь, учебные точки могут быть неравномерно распределены, и многие ячейки могут быть пустыми, как показано на рис.
Установление неоднородной сетки может привести к более сбалансированному расположению, но это затруднит быстрый поиск ячейки, содержащей q. Но также нет гарантии того, что ближайший сосед q фактически находится в той же ячейке, что и q, особенно если q, лежит очень близко к границе ячейки. Это означает, что мы должны обыскать и соседние ячейки, чтобы убедиться, что мы нашли абсолютного ближайшего соседа.



Линейная классификация

Будем рассматривать бинарную классификацию. В этом случае множество возможных значений ответов состоит из двух элементов.

$$Y = \{-1, +1\}$$

Иногда класс "+1" называют положительным, а класс "-1" – отрицательным.

Линейная модель классификатора

$$a(x) = in(w_0 + \sum_{j=1}^{d} w_j x^j)$$

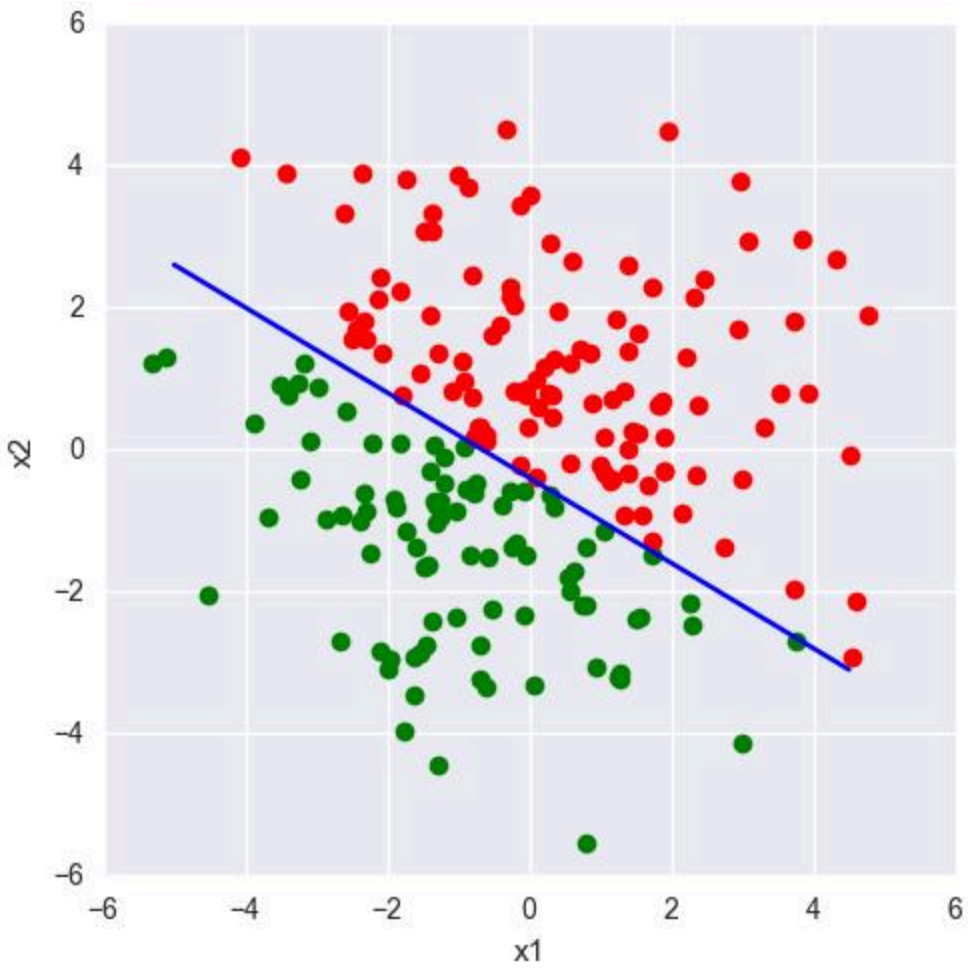
Замечание. Теоретически есть вариант, что под знаком будет стоять 0. Тогда либо можно решить, что это слишком редкий случай и на практике этого быть не может, либо можно договорится считать, что если сумма равна 0, знак от нее тоже будет равен нулю, и это означает отказ от классификации. То есть модель максимально не уверена в своем ответе, она вообще ничего не знает, и поэтому она не хочет делать вывода о том, положительный это или отрицательный класс.
Далее проделаем все те же шаги, что и для линейной регрессии:

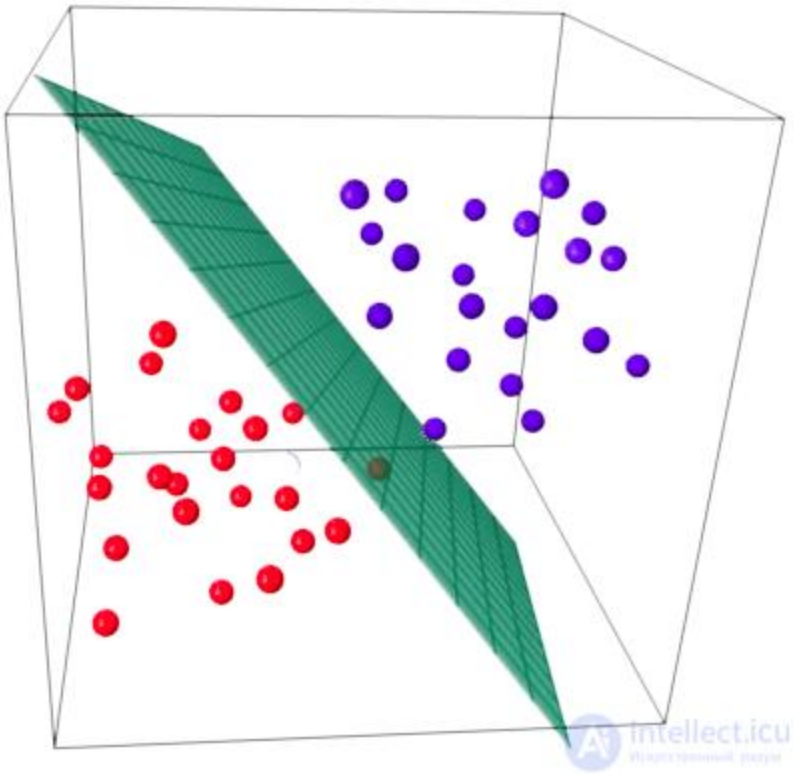
1. добавим единичный признак, то есть признак, который всегда равен единице
2. оставшуюся сумму запишем как скалярное произведение вектора весов w на вектор признаков x.
   Возьмем знак от этой суммы, получим линейный классификатор.

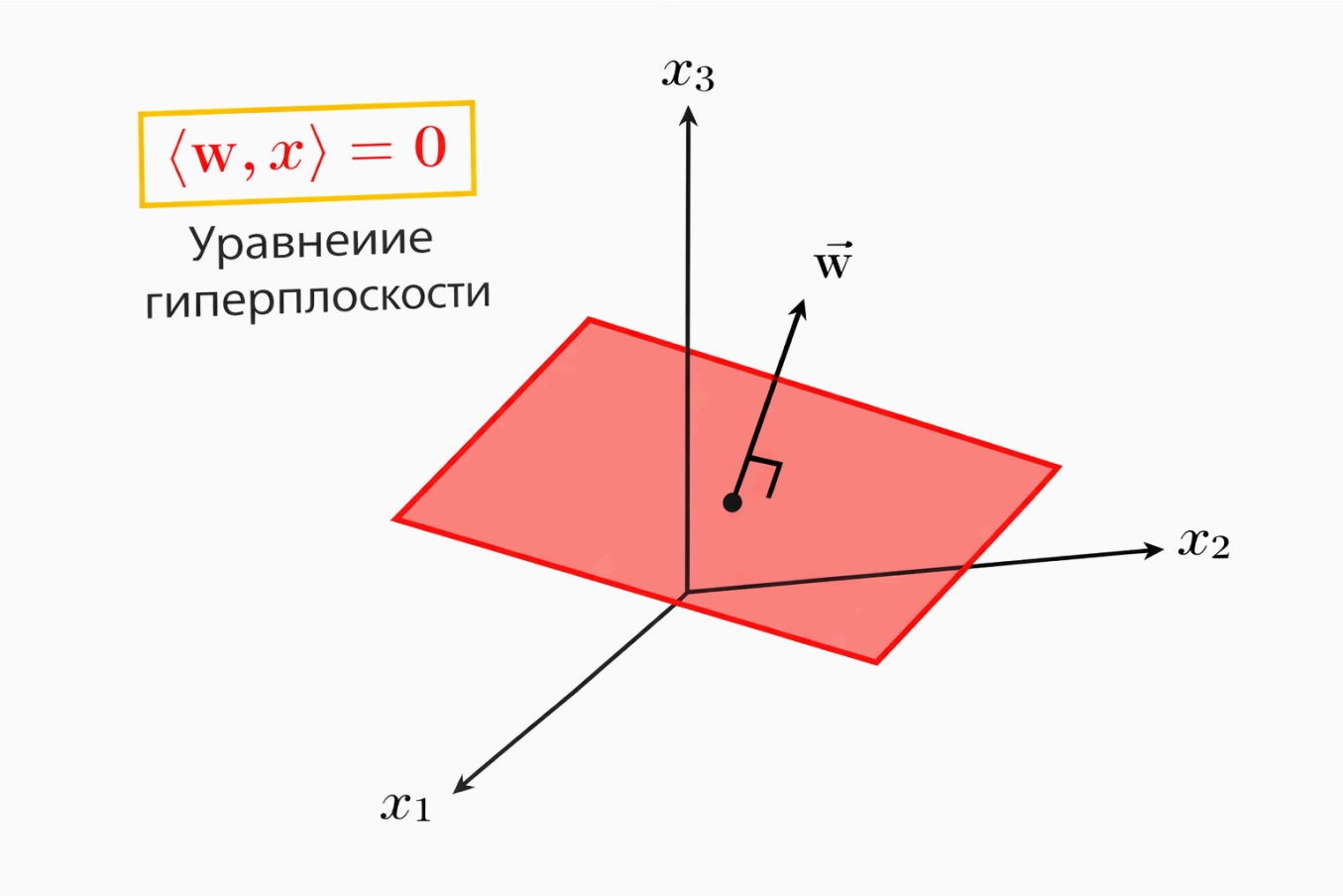
$$a(x) = in \sum_{j=1}^{d+1} w_j x^j = in < w, x >$$

Геометрическая интерпретация

Строится гиперплоскость, разделяющая классы. Например, если признаков 2, то строится прямая, если 3, то плоскость в трехмерном пространстве.







Уравнение гиперплоскости $< w, x >= 0$, w – вектор нормали

$< w, x >= 0$ – x лежит на гиперплоскости

$< w, x >< 0$ – x «слева» от нее

$< w, x >> 0$ – x «справа» от нее

Таким образом, линейный классификатор проводит плоскость в пространстве признаков и

относит объекты по разные стороны от нее к разным классам

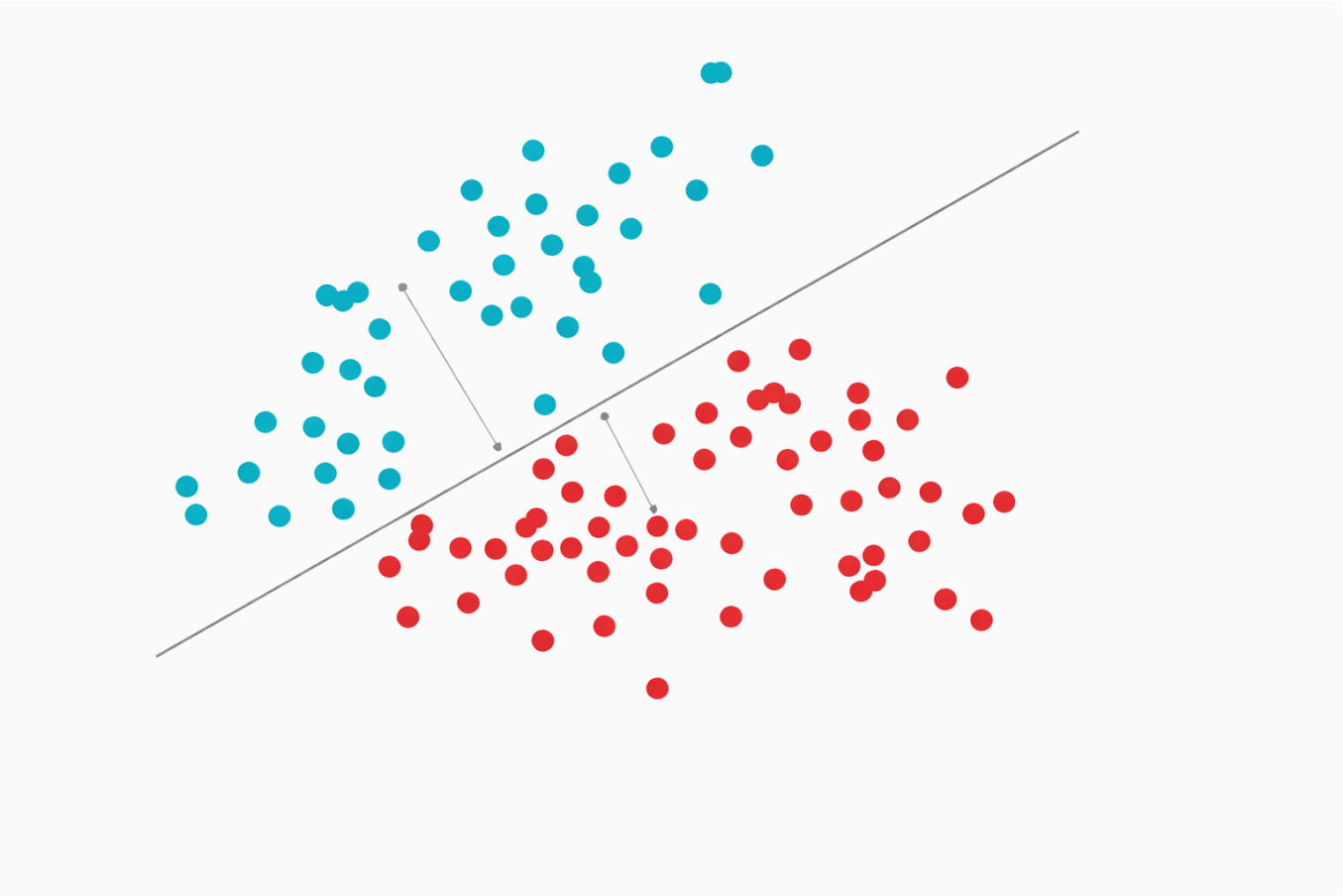
$< w, x >< 0$ – x относится к классу -1

$< w, x >> 0$ – x относится к классу 1

Расстояние от точки до гиперплоскости:

$< w, x >= 0$: $\frac{|<w,x>|}{||w||}$

Чем больше $< w, x >$, тем дальше объект от разделяющей гиперплоскости.



Когда модель ошибается?

1. когда $< w, x >> 0$, но объект из класса - 1
2. когда $< w, x >< 0$ но объект из класса 1

Иными словами, $y < w, x >< 0$.

$M_i = y_i < w, x_i >$

$M_i > 0$ — классификатор даёт верный ответ

$M_i < 0$— классификатор ошибается

Чем дальше отступ от нуля, тем больше уверенности.

Замечание. На самом деле формула для линейного классификатора в общем случае чуть сложнее. Нужно не просто брать знак от скалярного произведения, а сначала вычесть некоторое число (порог) из скалярного произведения и взять знак от этой разницы. Число называется порогом классификатора. Действительно, если скалярное произведение больше, чем t, объект должен быть объявлен положительным нашей моделью, а если меньше, чем , ,объект должен быть объявлен отрицательным.

То есть по умолчанию этот порог равен нулю, но мы можем его выбирать другим.
$a(x) = in(< w, x > -)$

Как измерять ошибку классификатора?

Простейший вариант: сравнить класс, который выдает модель, с классом, которому на самом деле принадлежит объект. Такая функция потерь называется бинарной. Это индикатор того, что прогноз не равен факту: если они не равны, то штраф - единица; если они равны, то штраф - ноль (по-английски это называется "zero-one", и эта функция потерь может принимать только 0 и 1)

$$L(ya(x)) = [a(x) \neq y]$$

На ее основе легко записать функционал ошибки. Это будет просто доля ошибок, то есть мы по всем объектам суммируем эти индикаторы и делим на количество объектов в выборке, получается доля объектов, на которых мы ошибаемся. Можно измерять не долю ошибок, а наоборот – долю верных ответов, в этом случае неравенство заменяется на равенство.

$$(a, X) = \frac{1}{l} \sum_{i=1}^{l} [a(x_i) \neq y_i]$$

Итак, возьмем получившийся функционал (доля ошибок), и внесем в него модель линейного классификатора.

$$(a, X) = \frac{1}{l} \sum_{i=1}^{l} [a(x_i) \neq y_i] = \frac{1}{l} \sum_{i=1}^{l} [in < w, x_i > \neq y_i] \to \min_{w}$$

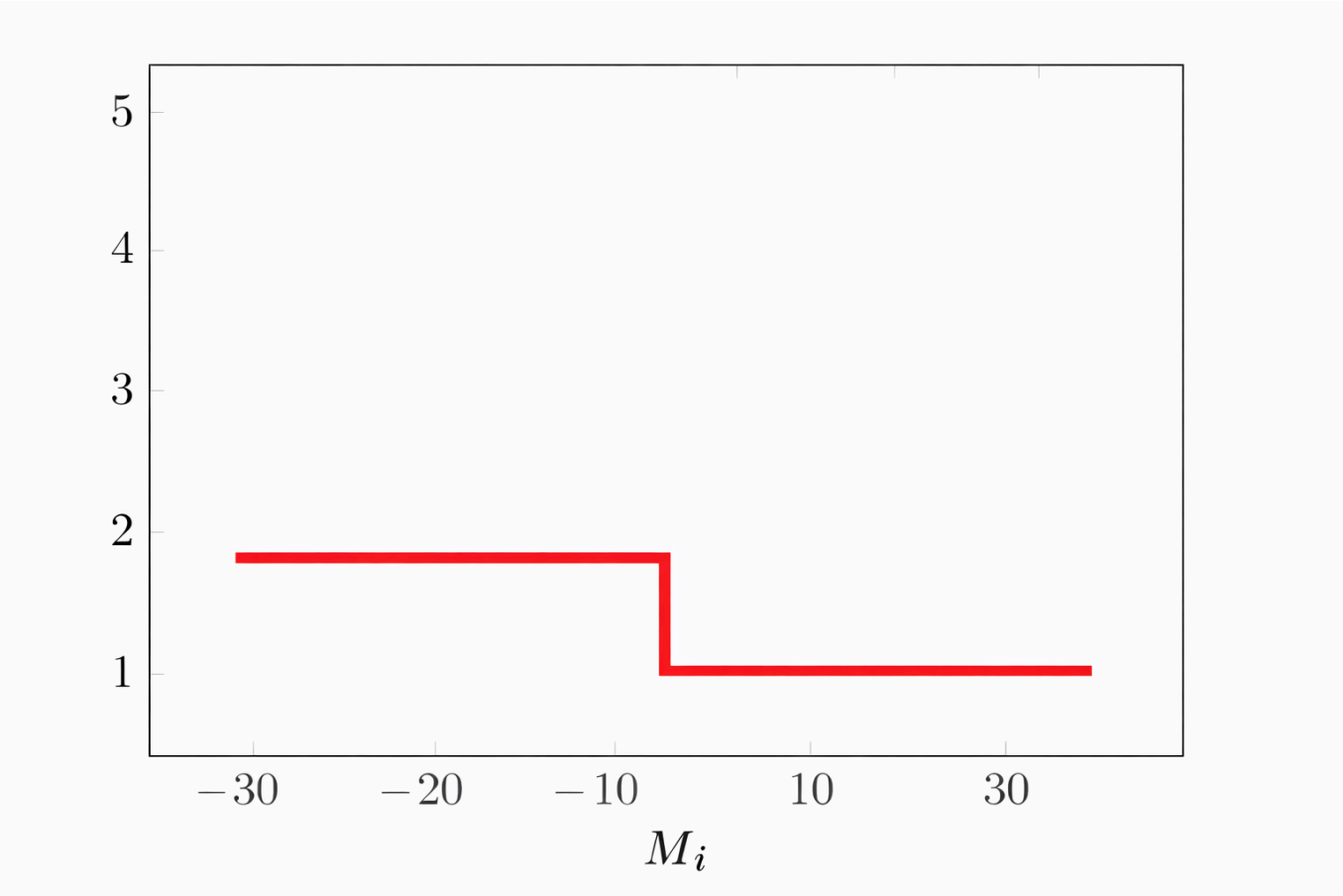
Какие здесь могут возникнуть проблемы?

Нельзя посчитать производную знака скалярного произведения по w, потому что функция дискретная.
Индикатор – тоже является дискретной функцией, и также довольно сложно будет посчитать производную.
Таким образом, градиентный спуск напрямую здесь не применим.
Можно заменить $in < w, x_i > \neq y_i$ на условие, что отступ на i-ом объекте меньше нуля.
Вспомним, что отступ меньше нуля только в том случае, если истинный ответ и прогноз модели имеют разные знаки. То есть это эквивалентная запись, но знака здесь нет.

$$(a, X) = \frac{1}{l} \sum_{i=1}^{l} [y_i < w, x_i > < 0]$$

где $y_i < w, x_i > = M_i$.
Функция потерь, зависит от отступа (по оси X – отступ, по оси Y – функция потерь).
Пороговая функция, которую нельзя продифференцировать.

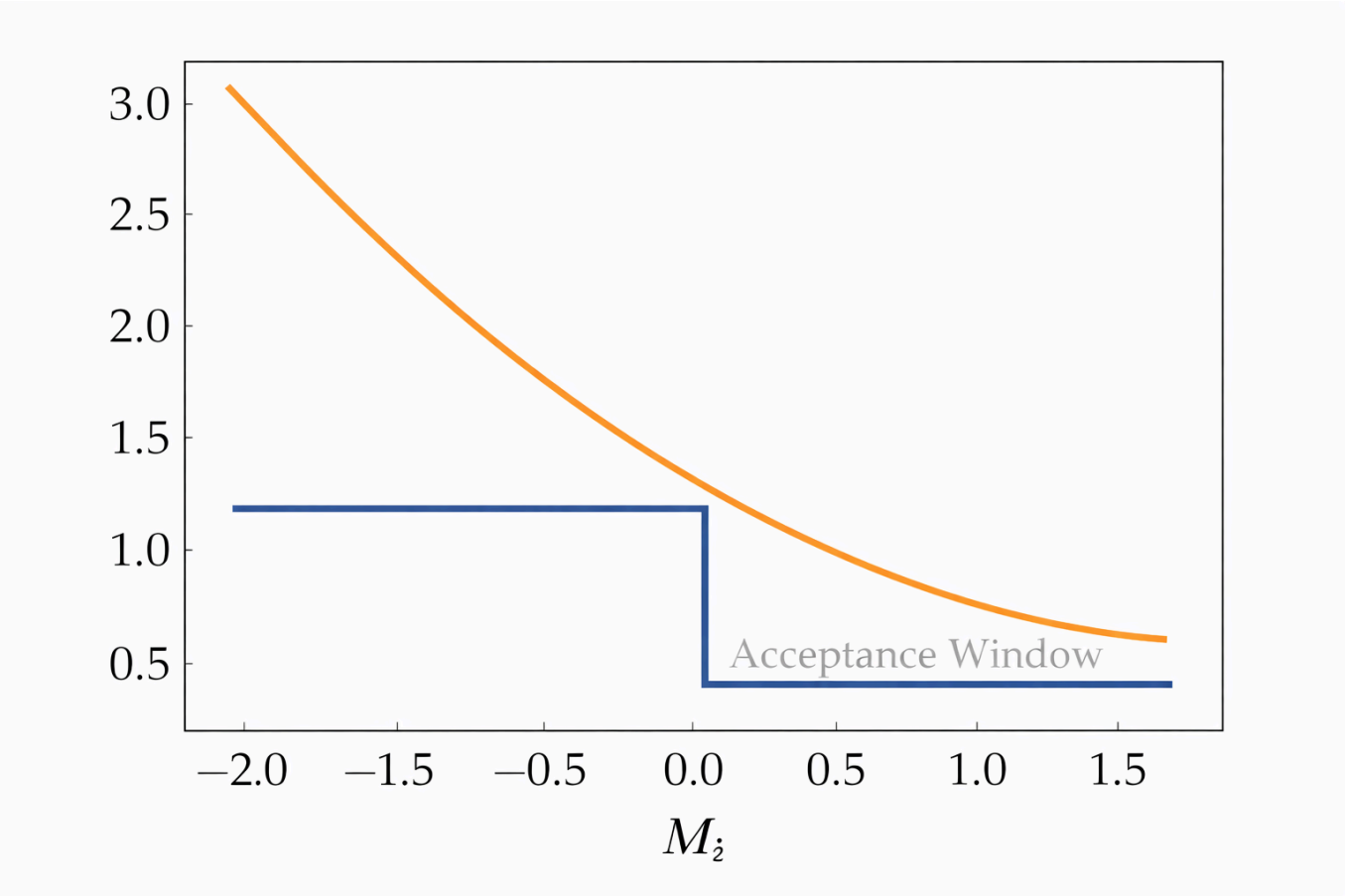
Допустим, мы находимся в точке - 1.0. Непонятно, куда шагать: влево или вправо. Влево - значение функции потерь 1, вправо - все еще значение 1. Производная в этой точке равна 0.



Как решить проблему?

Поменять функцию потерь так, чтобы знать, куда двигаться в каждой точке, при каждом значении отступа: влево или вправо, как менять отступ, чтобы ошибка уменьшилась.
Что для этого сделать? Хорошо бы как-то загнуть функцию потерь. То есть превратить прямые участки в дуги. Формально, идея следующая: найти некоторую верхнюю оценку, $L(M)$.
Это будет тоже функция потерь, которая ограничивает сверху нашу исходную бинарную функцию потерь. Оранжевый график удовлетворяет этому требованию - это какая-то Функция, которая лежит строго выше нашей бинарной функции потерь (в нуле они пересекаются). Будем тогда минимизировать эту верхнюю оценку (тогда она «придавит» долю ошибок к нулю, то есть, уменьшая эту правую часть неравенства, уменьшить и центральную часть неравенства)

$L(M) = [M < 0] <= L(M)$



$$(a, X) <= \ (a, X) = \frac{1}{l} \sum_{i=1}^{l} L(M_i) \to \min_a$$

Примеры оценок

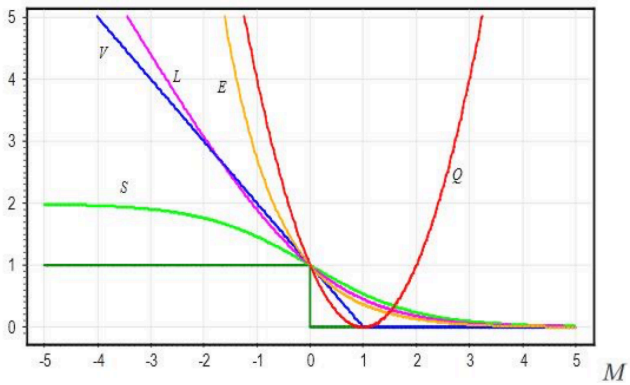


Рис. 7. Непрерывные аппроксимации пороговой функции потерь $[M < 0]$.

$Q(M) = (1 - M)^2$	— квадратичная;
$V(M) = (1 - M)_+$	— кусочно-линейная;
$S(M) = 2(1 + e^M)^{-1}$	— сигмоидная;
$L(M) = \log_2(1 + e^{-M})$	— логистическая;
$E(M) = e^{-M}$	— экспоненциальная.

$L(M) = (1 - M)+ = \max(0, 1 - M)$ – кусочно-линейная функция потерь (используется в методе опорных векторов)

$L(M) = (-M)+ = max(0, -M)$ – кусочно-линейная функция потерь (соответствует персептрону Розенблатта)

Замечания:

1. Кусочно-линейная функция (синяя) тоже не дифференцируемая, у нее есть особенная точка, единица (где угол), в которой нет производной. Но у этой функции есть наклон,

то есть если мы будем находиться левее единицы (то есть, мы ошибаемся), то хотя бы понятно куда двигаться, куда изменять отступ, чтобы ошибка уменьшилась. Метод опорных векторов базируется на этой функции потерь

2. Логистическая функция (розовая) соответствует логистической регрессии. Это метод является одним из вариантов линейной классификации

Рассмотрим логистическую функцию потерь (логарифм в логистической функции потерь можно брать с любым основанием, потому, что замена основания – это умножение на некоторую константу. Но лучше брать основание 2. Иначе может график может проходить чуть ниже и формально не быть верхней оценкой на пороговую функцию потерь)
Итак, имеем функционал ошибки, вычисляем его градиент по w, можем делать шаги градиентного спуска (так как теперь у нас функция потерь дифференцируемая).
Линейный классификатор, как и линейная регрессия, может переобучаться. В этом случае, чтобы понизить шансы переобучения, можно добавить штраф за сложность модели. Это точно такие же регуляризаторы, которые были для задачи регрессии – L2 норма (сумма квадратов весов) или L1 норма (сумма модулей весов).

$$(w, X) = \frac{1}{l} \sum_{i=1}^{l} \lg(1 + \mathrm{x}(-y_i < w, x_i >)) \to \min_w$$

Выбираем логистическую функцию потерь

$$(w, X) = \frac{1}{l} \sum_{i=1}^{l} \lg\left(1 + \mathrm{x}(-y_i w, x_i)\right) \to \min_w$$

Вычисляем градиент

$$_w\,(w, X) = -\frac{1}{l} \sum_{i=1}^{l} \frac{y_i x_i}{1 + \mathrm{x}(y_i w, x_i)}$$

Градиентный спуск

$$w^{()} = w^{(-1)} + \frac{1}{l} \sum_{i=1}^{l} \frac{y_i x_i}{1 + \mathrm{x}(y_i w, x_i)}$$

Пример регуляризации

$$(w, X) = \frac{1}{l} \sum_{i=1}^{l} \lg\left(1 + \mathrm{x}\left(-y_i w, x_i\right)\right) + w^2 \to \min_w$$

Пример

Дана тренировочная выборка из 5 объектов. Нужно построить линейный классификатор (разделяющую прямую для этих объектов).

Замечание. Без ошибки в данном случае этого сделать не получится.

	x	y
A	-2	-1
B	-1	1
C	0	-1
D	1	1
E	2	1

Отступы объектов равны:

$$M_1 = -1(-2w_1 + w_0)$$

$$M_2 = 1(-1w_1 + w_0)$$

$$M_3 = -1(0w_1 + w_0)$$

$$M = 1(1w_1 + w_0)$$

$$M_5 = 1(2w_1 + w_0)$$

В качестве мажорирующей функции возьмём $(M) = (1 - M)^2$.

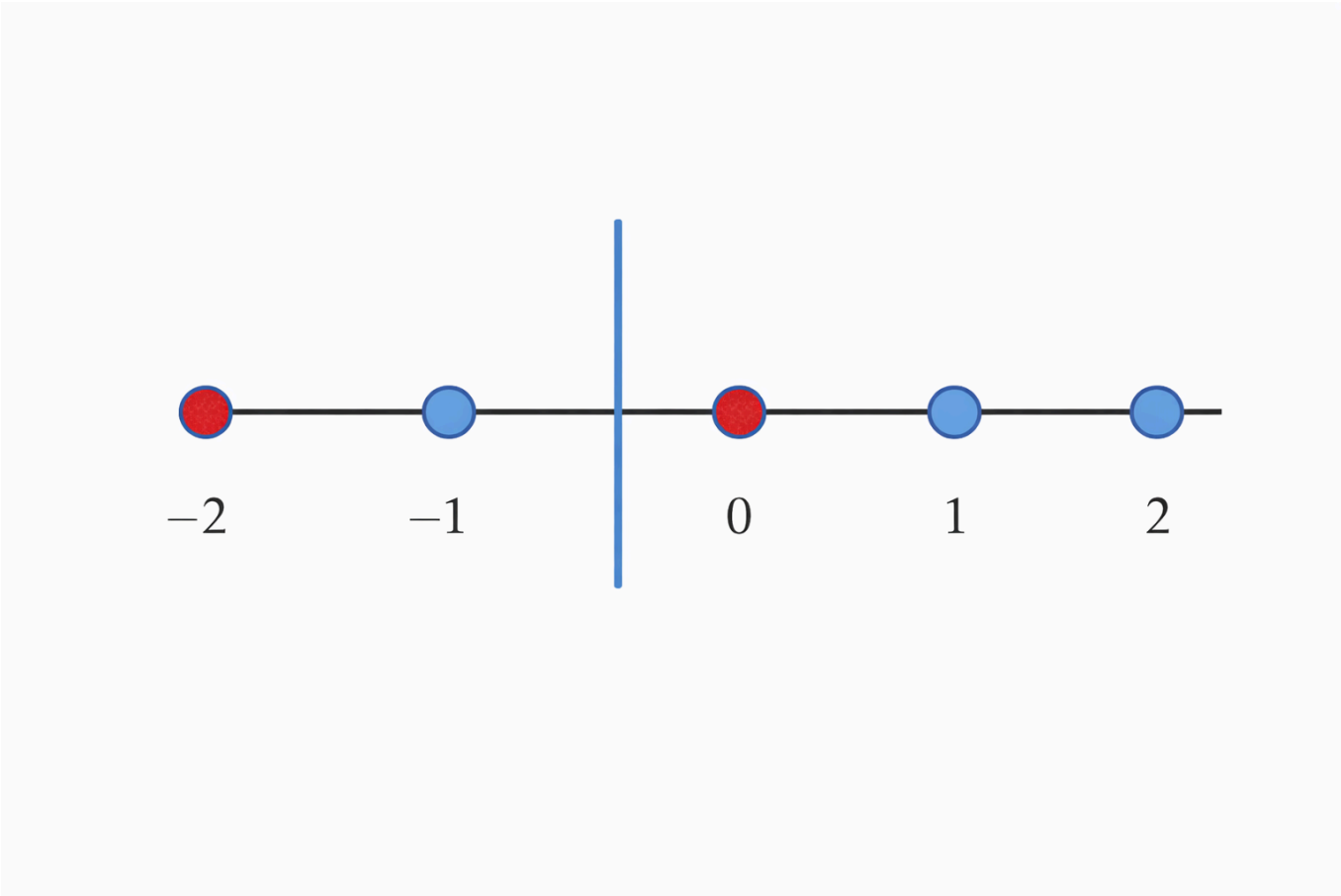
	x	y
A	-2	-1
B	-1	1
C	0	-1
D	1	1
E	2	1

$$L = (1 + (-2w_1 + w_0))^2 + (1 - (-1w_1 + w_0))^2 +$$
$$+(1 + 1(0w_1 + w_0))^2 + (1 - (1w_1 + w_0))^2 + (1 - (2w_1 + w_0))^2 =$$
$$= (1 - 2w_1 + w_0)^2 + (1 + w_1 - w_0)^2 + (1 + w_0)^2 + (1 - w_1 - w_0)^2 +$$
$$+(1 - 2w_1 - w_0)^2 = 10w_1^2 + 5w_0^2 - w_1 - 2w_0 + 5$$

$$\frac{L}{w_1} = 20w_1 - , \frac{L}{w_0} = 10w_0 - 2$$

Приравнивая производные к нулю, находим $w_1 = 0, w_0 = 02$
Следовательно, классификатор будет таким: если $0x + 02 > 0$ то объект из класса 1 (иначе из класса -1). Что эквивалентно: если $x > 05$, то объект из класса 1 (иначе из

класса - 1)



Метод градиентного спуска

$L = 10w_1^2 + 5w_0^2 - w_1 - 2w_0 + 5$

$$\frac{L}{w_1} = 20w_1 - , \frac{L}{w_0} = 10w_0 - 2$$

Градиент – это вектор ($20w_1 - , 10w_0 - 2$)
Пусть $(1, 1)$ – начало спуск. Зададим длину шага 0.05.

1. Подставляя $(1, 1)$ в градиент, получаем $(12,)$. Значит, нужно идти в направлении $(-12, -)$. Тогда $(1, 1) + 005\ \ (-12, -) = (0, 0)$
2. Подставляя $(0, 0)$ в градиент, получаем $(0,)$. Значит, нужно идти в направлении $(0, -)$. Тогда $(0, 0) + 005\ \ (0, -) = (0, 0)$
3. Подставляя $(0, 0)$ в градиент получаем $(0, 2)$. Значит нужно идти в направлении $(0, -2)$. Тогда $(0, 0) + 005\ \ (0, -2) = (0, 03)$.
4. Подставляя $(0, 03)$ в градиент получаем $(0, 1)$. Значит нужно идти в направлении $(0, -1)$. Тогда $(0, 03) + 005\ \ (0, -1) = (0, 025)$.
5. Подставляя $(0, 025)$ в градиент получаем $(0, 05)$. Значит нужно идти в направлении $(0, -05)$. Тогда $(0, 025) + 005\ \ (0, -05) = (0, 0225)$.

6. Подставляя $(0,0225)$ в градиент получаем $(0,025)$. Значит нужно идти в направлении $(0,-025)$. Тогда $(0,0225)+005\ \ (0,-025)=(0,02125)$.
7. Подставляя $(0,02125)$ в градиент получаем $(0,0125)$. Значит нужно идти в направлении $(0,-0125)$. Тогда $(0,02125)+005\ \ (0,-0125)=(0,02025)$.
8. Подставляя $(0,02025)$ в градиент получаем $(0,0025)$. Значит нужно идти в направлении $(0,-0025)$. Тогда $(0,02025)+005\ \ (0,-0025)=(0,0203125)$.
9. Подставляя $(0,0203125)$ в градиент получаем $(0,003125)$. Значит нужно идти в направлении $(0,-003125)$. Тогда $(0,0203125)+005\ \ (0,-003125)=(0,0201525)$.

Видно, что сходится к $(0,02)$.

Д/з

1. Выписать определения и формулы метрик из лекций (Коэффициент корреляции, косинусная мера, коэффициент Жаккара, редакционное расстояние, расстояние Левенштейна, расстояния Хемминга)
2. Для каких типов признаков можно использовать эти меры близости?
3. Когда нормализация ухудшает результаты для kNN (необязательно).
4. Попробуйте дать теоретическую оценку ($O(n)$, $O(n^2)$) использования данных (диаграммы вороного, Kd-деревья методов при использовании в kNN. Дополнительно: Локальное хеширование, R-деревья, дерево квадрантов
5. Прочитайте материалы по использованию метода KNN для классификации изображений. Как в этих случаях определяется расстояние? https://towardsdatascience.com/scanned-digits-recognition-using-k-nearest-neighbor-k-nn-d1a1528f0dea, https://towardsdatascience.com/mnist-with-k-nearest-neighbors-8f6e7003fab7, https://www.reg.ru/blog/stehnfordskij-kurs-Jokaiva-2.kascifikaciva-izohrzhonii/ (на хорошую оценку)
6. Дана функция: $(u,)=u^2-2^2-2u++1$. Будем искать точку минимума с помощью метода градиентного спуска. Длину шага полагаем равной 0,5. В какую точку мы попадем из точки (0,0) после одной итерации метода градиентного спуска? После двух итераций?